

Revisiting Long-Tailed Learning: Insights from an Architectural Perspective

Yuhan Pan
University of Science and Technology
of China
Hefei, China
yuhanpan@mail.ustc.edu.cn

Yanan Sun
Sichuan University
Chengdu, China
ysun@scu.edu.cn

Wei Gong*
University of Science and Technology
of China
Hefei, China
weigong@ustc.edu.cn

Abstract

Long-Tailed (LT) recognition has been widely studied to tackle the challenge of imbalanced data distributions in real-world applications. However, the design of neural architectures for LT settings has received limited attention, despite evidence showing that architecture choices can substantially affect performance. This paper aims to bridge the gap between LT challenges and neural network design by providing an in-depth analysis of how various architectures influence LT performance. Specifically, we systematically examine the effects of key network components on LT handling, such as topology, convolutions, and activation functions. Based on these observations, we propose two convolutional operations optimized for improved performance. Recognizing that operation interactions are also crucial to network effectiveness, we apply Neural Architecture Search (NAS) to facilitate efficient exploration. We propose LT-DARTS, a NAS method with a novel search space and search strategy specifically designed for LT data. Experimental results demonstrate that our approach consistently outperforms existing architectures across multiple LT datasets, achieving parameter-efficient, state-of-the-art results when integrated with current LT methods.

CCS Concepts

• Computing methodologies → Neural networks.

Keywords

Neural Architecture Search, Image Classification, Long-Tailed Learning

ACM Reference Format:

Yuhan Pan, Yanan Sun, and Wei Gong. 2025. Revisiting Long-Tailed Learning: Insights from an Architectural Perspective. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25)*, November 10–14, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3746252.3761191>

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIKM '25, Seoul, Republic of Korea.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2040-6/2025/11
<https://doi.org/10.1145/3746252.3761191>

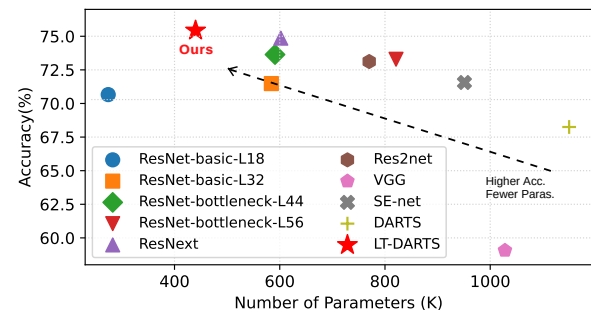


Figure 1: The performance of different architectures on the long-tailed CIFAR-10-LT dataset. The direction of black dashed lines indicates better architectures, as they achieve higher accuracy with fewer architectural parameters.

1 Introduction

Although deep neural networks have been proven effective in solving many problems [12, 15, 32, 52], their robustness under imbalanced data distributions still requires further investigation. In reality, data distributions in many practical scenarios are Long-Tailed (LT) [4, 28, 37], characterized by a few head classes with abundant samples and many tail classes with limited samples. It has been proved that conventional deep learning methods tend to exhibit sub-optimal performance in such a context. To address this challenge, the community has proposed numerous approaches including data augmentation [1, 19, 22, 25], class re-sampling strategies [3, 26], decoupling learning [23, 51] and so on. Nevertheless, most existing methods pay insufficient attention to neural architecture design, which is crucial for deep models to achieve optimal performance.

It is widely recognized that meticulous architectural design contributes to improved performance on balanced data distribution, and this consensus unsurprisingly extends to LT data distribution. As depicted in Fig. 1, we conduct extensive training on several architectures and report their parameter sizes and performance. It can be observed that different backbones lead to various performances and an optimal architecture may achieve superior performance even with fewer parameters, which validates the feasibility of approaching the LT problem from an architectural perspective.

While neural architectures have been extensively evaluated on balanced datasets, it remains unclear *whether these architectures generalize effectively to LT distributions*. To explore this, we conduct a preliminary study. We randomly sample 10 architectures

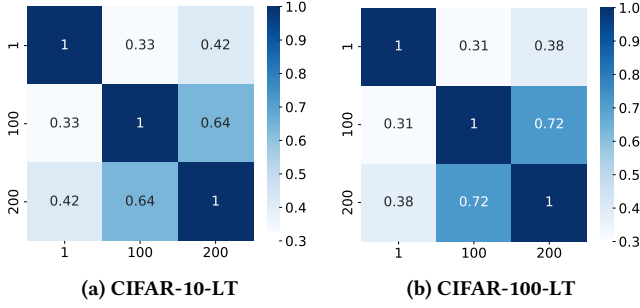


Figure 2: Heatmap of Kendall's Tau correlation under different imbalance factors. A lower tau (lighter color) indicates weaker performance correlation.

from NAS-Bench-201 [10], a benchmark comprising 15,625 candidate networks. Each architecture is evaluated under three class imbalance settings with imbalance factors $\rho \in \{1, 100, 200\}$, where $\rho = n_{max}/n_{min}$ denotes the ratio of the largest to smallest class sizes. For each setting, we record accuracy-based rankings of the architectures. We then compute Kendall's Tau correlation between the rankings across different imbalance levels (Figure 2). Results show weak correlations, particularly between balanced and highly imbalanced settings, indicating that architectures performing well under balanced conditions may not retain their performance in LT scenarios. These findings suggest that architecture performance is not invariant to class distribution, motivating the need to design and search for architectures explicitly suited to long-tailed data.

To this end, we first conduct a systematic analysis of key architectural components under LT data, including network topology, activation functions, convolutional operations, and normalization methods. Our study reveals several design patterns that consistently improve performance in LT settings. Building on these insights, we introduce two new convolutional operations, named LT-AggConv and LT-HierConv, tailored for LT scenarios. Since the arrangement and interaction of operations also play a critical role in model performance, we further propose LT-DARTS, a neural architecture search (NAS) algorithm with an LT-aware search space and an unbiased search strategy, enabling efficient discovery of effective architectures. Extensive experiments show that our method mitigates LT issues at the architectural level. Moreover, it is orthogonal to existing approaches and can be easily combined with them to achieve state-of-the-art results. Our contributions can be concluded as the following:

- We bridge the gap between neural architecture and LT. Through extensive experimentation, we identify the architectural components that perform better in LT scenarios and propose LT-AggConv and LT-HierConv, two convolutions specifically designed for LT.
- We propose LT-DARTS, including a search space better suited for LT and a de-biasing search strategy, to explore the use of NAS in developing an LT-friendly architecture.
- Extensive experiments show that the architectures we discover outperform crafted designed architectures. And it can seamlessly integrate with existing solutions tailored to LT,

further enhancing model performance. Our research provides a complementary perspective for the LT community.

2 Related Work

2.1 Deep Long-tailed Learning

Recently much effort has been devoted to deep long-tailed recognition. A branch of works [5, 18, 47, 50] tackle the LT problem from the perspective of class re-balance, proposing novel sampling strategies and loss functions to effectively re-weight imbalanced samples. Meanwhile, data augmentation is also introduced to mitigate this issue [1, 33]. They explore how to enhance the representation of minority classes through data processing. Moreover, some studies [23, 38, 49] decouple the feature representation and classifier, improving each module independently, which effectively enhanced the performance of the model on LT data. However, existing methods have paid little attention to the influence of network architecture on the long-tailed problem, as most of them directly adopt popular backbone networks such as ResNet [16] for feature extraction. This paper takes a different approach by focusing on the design of the network, providing a complementary perspective for LT.

2.2 Differentiable Architecture Search

As one of the representative methods of NAS, Differentiable Architecture Search (DARTS) has garnered significant attention in recent years because of its powerful performance and reduced computational requirements. Liu et al. pioneer the introduction of DARTS [27], allowing an efficient search of the architecture using gradient descent. Subsequently, a series of variations have been proposed to optimize the vanilla version. PC-DARTS [41] achieves higher efficiency by randomly sampling a subset of channels for operation search. R-DARTS [45] proposes an early stopping criterion to make the searched model more robust. Fair-DARTS [7] relaxes the exclusive competition between skip-connections and achieves higher performance. Although these works have improved DARTS to some extent, their settings are still based on uniform data distribution. In this paper, we introduce DARTS to LT and propose targeted improvements to address its limitations under LT, allowing it to search for architectures with enhanced performance.

3 Investigation of Components for LT

To design a LT-friendly architecture, we first conduct extensive experiments to explore the performance of architectural components under LT distributions. As shown in Fig. 3, our explorations cover various aspects, including topology, the specific design of convolutions, the selection and placement of activation functions, and different normalization methods. Based on experimental observations, we distill the key components suited for LT data and propose two new convolutional architectures: LT-AggConv and LT-HierConv.

3.1 Preliminaries

We initially introduce the experimental setup for exploring LT-friendly networks and the evaluation metric.

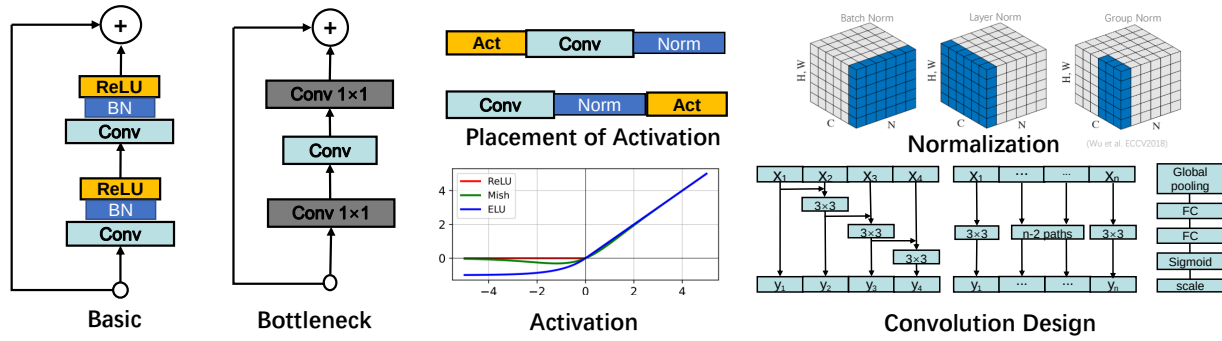


Figure 3: An overview of the architectural properties we explore. “Basic” represents the foundational architecture. “Bottleneck” refers to an improved topology, with the activation functions and normalization methods for each layer omitted for simplicity. Additionally, we investigate the design of convolution, placement of activation, normalization, and activation.

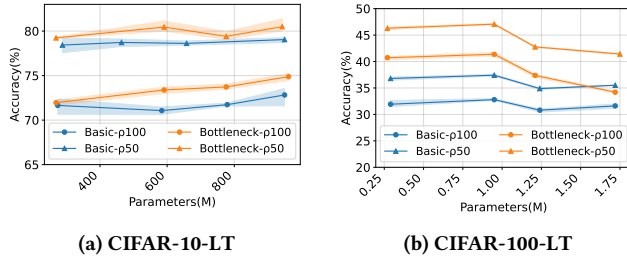


Figure 4: Comparison between “Basic” and “Bottleneck”, and ρ denotes the imbalance ratio.

Basic architecture. The current LT community universally uses ResNet series networks as the backbone for feature extraction [18, 34, 36, 49]. Without loss of generality, our exploration is also based on ResNet [16]. The basic architecture consists of three network layers, each comprising several stacked blocks. The block design of the baseline follows the ResNet basic block structure, where each block is composed of two identical convolution-BN-ReLU modules and a shortcut connection.

Training. We employ the standard cross-entropy loss and set the batch size to 128. The optimization uses the SGD optimizer with a learning rate of 0.1, a momentum of 0.9, and a weight decay $2e-4$. These settings are chosen to ensure an appropriate balance between effective learning dynamics and regularization.

Evaluation. We evaluate the performance of individual components on the CIFAR-10-LT and CIFAR-100-LT datasets. These two datasets are variations of the vanilla CIFAR dataset, sampled with different quantities across classes to create LT datasets.

3.2 Topology

We explore the performance of basic and bottleneck topology firstly, which are proposed in [16]. While the bottleneck has been proven to perform better on balanced datasets, many existing studies in the LT community default to using the basic topology in the backbone network [5, 51]. We evaluate the network with basic and bottleneck topology on the LT dataset. The experimental results in Fig 4 demonstrate that the bottleneck topology outperforms the

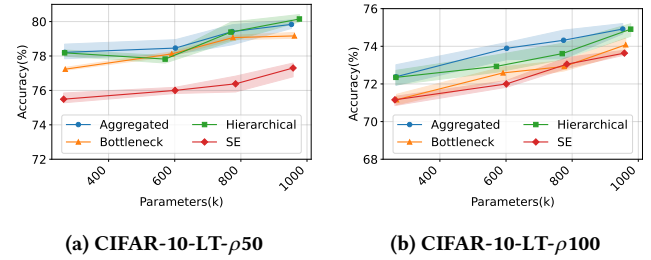


Figure 5: Performance of different convolutions on CIFAR-10-LT with $\rho = \{50, 100\}$.

basic, which aligns with the balanced context and suggests that the bottleneck may be a better choice in the search space.

3.3 Specific Convolution Design

The specific design of convolutions primarily determines the network’s ability to feature representation. In addition to the basic convolution operation (where we use the bottleneck topology), we further explore aggregate convolution [40], hierarchical convolution [13], and squeeze and excitation (SE) convolution [20], which are all proven effective on the balanced dataset. As shown in Fig. 3, the aggregate architecture extends the single path of conventional convolution into multiple paths, while the hierarchical architecture incorporates several sub-paths within the convolution to propagate feature information. The SE module dynamically adjusts the weight of each channel, enhancing the model’s ability to focus on important features. These optimized convolutions have been proven efficient on balanced datasets, and we observe their performance on LT datasets. Fig. 5 shows that both the aggregated and hierarchical architectures enhance performance on LT data. However, the SE module leads to a performance drop, unlike its behavior on balanced datasets.

3.4 Placement of Activation

According to [17], the placement of the activation function also impacts model performance. We conduct experiments to explore the

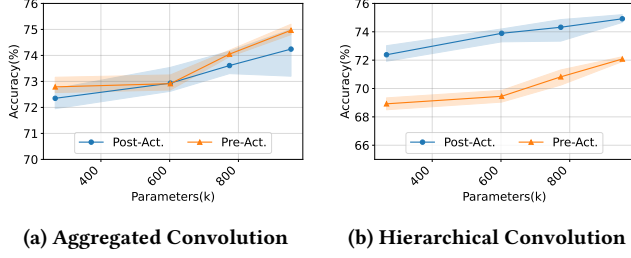


Figure 6: Investigating the impact of activation function placement on the performance of aggregated convolution and hierarchical convolution.

optimal position of the activation function under different convolution types. Since the SE module has been shown to underperform compared to basic convolution, we focus only on the aggregated and hierarchical convolutions. Specifically, we examine two methods: pre-activation and post-activation. The results, presented in Fig. 6, show that pre-activation benefits aggregation convolution, while post-activation is more effective for hierarchical convolution.

3.5 Activation

We study three different activation functions: ReLU [14], ELU [8], and Mish [29]. ReLU is highly efficient and widely used in many deep-learning models. ELU (Exponential Linear Unit) improves upon ReLU by allowing negative values, which helps mitigate the issue of dying neurons and can accelerate convergence. Mish, a newer activation function, combines the benefits of both ReLU and ELU, offering smoother gradients and improving model performance, particularly in deeper networks. We evaluate the performance of these activation functions in the context of LT data to determine which one provides the best results. Fig. 7a reveals that ReLU remains the superior choice.

3.6 Normalization

We investigate three normalization methods: BatchNorm [21], GroupNorm [39], and LayerNorm [2], to explore their performance on long-tailed datasets. As shown in Fig. 7b, batch normalization achieves the best performance under comparable model sizes. This suggests that BatchNorm’s ability to stabilize training by normalizing feature distributions per batch is particularly effective in mitigating the challenges posed by class imbalance in LT data. Therefore, we employ BatchNorm when constructing convolutional operations.

3.7 Operations Design

Based on these observations, we design two convolutional operations specifically tailored for LT issues: LT-AggConv and LT-HierConv. The design details of the two novel convolutional operations are depicted in Fig. 8. LT-AggConv employs an aggregated convolution with a pre-activation approach, which uses multiple parallel convolutional operations and combines their outputs. This design benefits LT scenarios by providing more refined feature representations for rare classes. LT-HierConv, on the other hand, uses

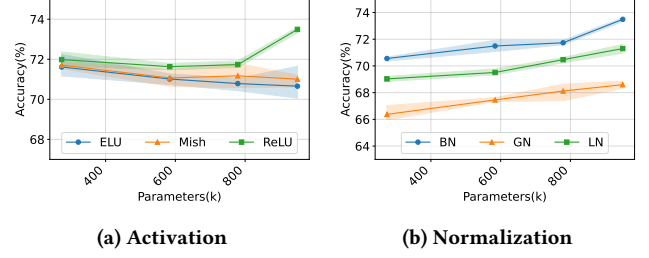


Figure 7: (a) Performance of different activation functions on CIFAR-10-LT. (b) Impact of different normalization methods on model performance.

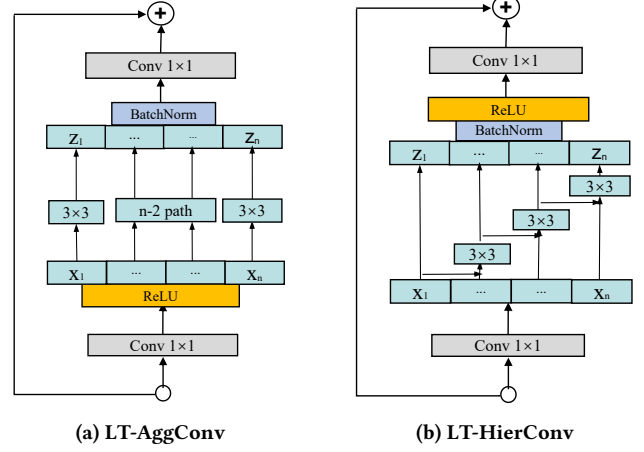


Figure 8: Architecture of proposed LT-AggConv and LT-HierConv operators.

Table 1: Performance comparison of LT-Res with existing ResNet series networks on long-tailed data.

Backbone	#P(M)	CIFAR-100-LT		
		$\rho=200$	$\rho=100$	$\rho=50$
ResNet	0.46	34.75 $\pm$ 0.25	38.38 $\pm$ 0.18	43.72 $\pm$ 0.17
ResNeXt	0.45	36.40 $\pm$ 0.26	41.14 $\pm$ 0.15	46.52 $\pm$ 0.28
Res2Net	0.45	35.84 $\pm$ 0.33	40.18 $\pm$ 0.22	46.33 $\pm$ 0.21
LT-Res	0.45	39.71$\pm$0.14	41.65$\pm$0.21	47.04$\pm$0.27

hierarchical convolution with a post-activation approach, allowing the model to adaptively emphasize relevant features at various hierarchical levels, thereby enhancing classification accuracy. Both operations utilize the ReLU activation function and BatchNorm normalization based on a bottleneck topology.

To demonstrate the superiority of the proposed approach, we craft an LT-friendly residual network based on the ResNet prototype, named LT-Res. Specifically, we replace the convolution operations in ResNet with LT-AggConv and LT-HierConv and construct the network according to the base architecture outlined in Section 3.1. To highlight the effectiveness of LT-Res, we compare it against ResNet, ResNeXt, and Res2Net. As shown in Table 1,

performance gains are achieved through architecture optimization alone.

4 A NAS Method for LT: LT-DARTS

To design networks better suited for LT, we propose LT-DARTS, a straightforward yet effective NAS method optimized for LT challenges. Although our proposed LT convolutions demonstrate effectiveness, previous study highlights that the interconnections among architectural components are also crucial [48]. Exhaustively exploring these connectivity patterns can be resource-intensive; thus, we leverage NAS to efficiently address this complexity. Specifically, we enhance DARTS [27], a gradient-based NAS approach, with targeted improvements in both the search space and strategy, leading to the development of LT-DARTS.

4.1 Preliminaries

DARTS constructs the neural network by stacking cells with the same architecture, where a cell is defined as a directed acyclic graph (DAG). The goal of search process is to identify the operations (edges) utilized for connecting diverse features (nodes). To this end, DARTS makes use of continuous relaxation in the following form:

$$\bar{o}(x) = \sum_{o \in \mathcal{O}} \frac{\exp(\alpha_o)}{\sum_{o' \in \mathcal{O}} \exp(\alpha_{o'})} o(x) \quad (1)$$

where $\mathcal{O}$ is the search space and $o(\cdot)$ is the different operations selected from $\mathcal{O}$. $\bar{o}(\cdot)$ is the mixed operation, and it will be replaced by the $o(\cdot) = \arg\max_{o \in \mathcal{O}} \alpha_o$ at the end of the search. α is a vector of dimension $|\mathcal{O}|$, named architecture parameter, denotes the operation mixing weights. It can be optimized through the following steps:

$$\min_{\alpha} \mathcal{L}_{val}(w^*(\alpha), \alpha) \quad (2)$$

$$s.t. \quad w^*(\alpha) = \arg\min_w \mathcal{L}_{train}(w, \alpha) \quad (3)$$

where w denotes the network parameters and $w^*(\alpha)$ is the optimal network parameters w under the current architecture α . $\mathcal{L}_{train}$ and $\mathcal{L}_{val}$ denote the training and validation losses, respectively. The search process is based on gradient optimization of α and w to find the optimal cell architecture.

4.2 Search Space towards LT-DARTS

We construct an LT-friendly search space, denoted as $\mathcal{O}$. Following the previous approaches [41, 43, 44], we include operations like 3×3 max pooling, 3×3 average pooling, residual connections, and zero (representing no connection) within the search space. Uniquely, based on our observations, we additionally introduce LT-AggConv and LT-HierConv as candidate convolution operations, each available with kernel sizes of $\{3 \times 3, 5 \times 5, 7 \times 7\}$. Other configuration settings follow those established by [27]. Each cell in our settings consists of 7 nodes: two input nodes, four intermediate nodes, and one output node. The output node serves as an aggregation of all other nodes within the cell. We form the network by stacking multiple cells, setting the first and second nodes of cell k equal to the outputs of cells $k-2$ and $k-1$, respectively. To enhance computational efficiency, we insert a reduction cell at $2/3$ of the network depth, where all operations adjacent to the input nodes use a stride of 2.

4.3 Search Strategy towards LT-DARTS

To the best of our knowledge, no previous work has investigated the performance of DARTS on LT. In light of this, we conduct a systematic study and find that directly applying the vanilla DARTS to LT leads to suboptimal performance, as the linear classifier introduces bias during the search process. To overcome this issue, we introduce a new classifier with frozen parameters, updating only the backbone weights during the search process. This approach mitigates the bias introduced by the classifier and facilitates the discovery of more effective architectures.

4.3.1 Observation: A Biased Search. Previous studies have identified classifier bias in LT model training [23, 31]. Similarly, we observe a gradual shift in the classifier during the network search process. As shown in Fig. 9, on a balanced dataset, inter-class weight norms remain similar and show little change during the search process. In contrast, the differences in inter-class norms increase as the search progresses on LT data. Additionally, in the final epoch, the angles between weight vectors become more unevenly distributed on LT data compared to balanced data. Such biased norms and angles lead to a biased search, where head classes dominate the loss gradients, causing the learned architectures to overfit features that dispropert.

4.3.2 Balanced Fixed Classifier for Search Decoupling. To address the bias search process, we introduce a Balanced Fixed Classifier (BFC) into the NAS process, a static, class-agnostic linear layer designed to decouple the architecture search from class frequency bias. This design draws inspiration from the phenomenon of neural collapse [42], where well-trained networks tend to converge toward a configuration where feature means and classifier weights form an Equiangular Tight Frame (ETF). Rather than treating this as a post-hoc outcome, we adopt this geometric structure as a prior and impose it from the start of the search.

Formally, the classifier weights $\mathbf{W} \in \mathbb{R}^{d \times C}$ are constructed to have equal norm and mutual angles, forming a centered ETF:

$$\mathbf{W} = \sqrt{E_W \frac{C}{C-1}} \mathbf{U} \left(\mathbf{I} - \frac{1}{C} \mathbf{1}\mathbf{1}^T \right) \quad (4)$$

where $\mathbf{U} \in \mathbb{R}^{d \times C}$ is an orthonormal basis satisfying $\mathbf{U}^T \mathbf{U} = \mathbf{I}$, and E_W controls the weight scale. The ETF structure ensures that each class occupies an equal and uniformly distributed position in the weight space, regardless of its sample count.

Crucially, we freeze the classifier throughout the architecture search, updating only the backbone parameters. This prevents class imbalance from distorting the gradient flow during optimization, effectively neutralizing the dominance of head-class signals. As a result, the search process becomes more balanced and fair, encouraging the discovery of architectures that are not just optimized for frequent classes, but are also robust to tail-class sparsity.

4.4 LT-DARTS Algorithm Overview

We formalize the overall execution of the proposed LT-DARTS. Compared with vanilla DARTS, LT-DARTS incorporates two key modifications: (1) an LT-aware search space that includes LT-AggConv and LT-HierConv operators, and (2) a search strategy with a BFC

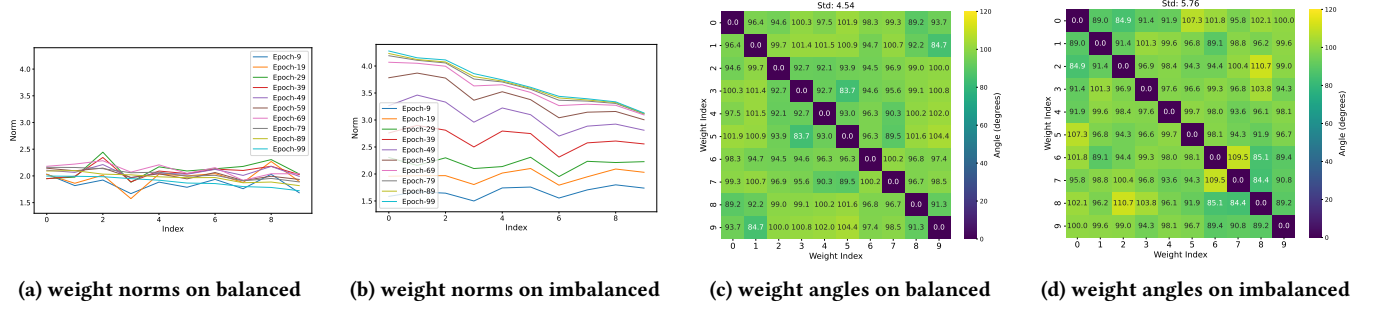


Figure 9: Classifier weight biased phenomena. (a) and (b) show weight norms at different search epochs, highlighting shifts in weight norms with long-tailed data. (c) and (d) reveal that the angle distribution between weight vectors is more uneven under long-tailed distributions. Best viewed in color.

to mitigate bias during optimization. The BFC adopts an ETF structure and remains frozen throughout the search, ensuring fairer gradient updates across head and tail classes. The search alternates between updating network weights w using the training set $\mathcal{D}_{\text{train}}$ and updating architecture parameters α using the validation set $\mathcal{D}_{\text{val}}$. After T epochs, the final discrete architecture is obtained by selecting the strongest operation on each edge. The full procedure is described in Algorithm 1.

Algorithm 1: LT-DARTS

Input: Training data $\mathcal{D}_{\text{train}}$, Validation data $\mathcal{D}_{\text{val}}$, Search space $\mathcal{O}$, Total epochs T , Network $f_{\alpha, w}$
Output: Final discrete architecture $\mathcal{A}$
Initialize network weights w and architecture parameters α ;
Construct a balanced fixed classifier $\mathbf{W}_{\text{BFC}}$ as Eq. 4
for $t = 1$ **to** T **do**
 Sample mini-batches $\mathcal{B}_{\text{train}} \subset \mathcal{D}_{\text{train}}$, $\mathcal{B}_{\text{val}} \subset \mathcal{D}_{\text{val}}$;
 // Update network weights w
 Compute training loss:
 $\mathcal{L}_{\text{train}} = \text{CE}(f_{\alpha, w}(\mathcal{B}_{\text{train}}), \mathbf{W}_{\text{BFC}})$;
 Update $w \leftarrow w - \eta_w \nabla_w \mathcal{L}_{\text{train}}$;
 // Update architecture parameters α
 Compute validation loss: $\mathcal{L}_{\text{val}} = \text{CE}(f_{\alpha, w}(\mathcal{B}_{\text{val}}), \mathbf{W}_{\text{BFC}})$;
 Update $\alpha \leftarrow \alpha - \eta_{\alpha} \nabla_{\alpha} \mathcal{L}_{\text{val}}$;
 // Discretize architecture
 foreach edge (i, j) in DAG cell **do**
 Select $o^* = \arg \max_{o \in \mathcal{O}} \alpha_o^{(i, j)}$;
 Add o^* to architecture $\mathcal{A}$
return $\mathcal{A}$

5 Experiments

In this section, we evaluate the performance of architectures discovered by the proposed LT-DARTS. We compare with state-of-the-art methods on various LT datasets and conduct ablation experiments to demonstrate the effectiveness of our approach.

5.1 Dataset and Implementation

We evaluate the model’s performance across four datasets of varying scales: CIFAR10-LT, CIFAR100-LT, Places-LT, ImageNet-LT, and iNaturalist-LT. Without loss of generality, we apply different imbalanced factors to CIFAR datasets. We compare our method with some state-of-the-art approaches including LDAM-DRW [5], BBN [50], Balanced Softmax [34], MiSLAS [49], LADE [18], GCL [24], RSG [36], ResLT [9], DisAlign [47] and BGP [35].

Our framework is implemented using PyTorch v1.13.0 and CUDA v11.7. For the specific details of LT-DARTS, we mostly followed the default settings of DARTS but made changes to certain parameters. Specifically, we use a smaller number of proxy layers (6 layers) to accelerate the search process and impose a larger weight decay $\mu = 6e-4$ to enhance the generalizability of the model. For the comparison subjects lacking experimental results, we reproduce them following the default settings of their respective open-source codes.

5.2 Main Results

5.2.1 Improvements Brought by Architectural Changes Only. Firstly, we compare the performance of the best architecture discovered by LT-DARTS with ResNet, ResNeXt, and Res2Net on the CIFAR-10/100-LT datasets. It should be noted that Vision Transformer (ViT)[11], which typically involve much larger parameter counts, have not been widely adopted as baselines in previous LT learning research. Therefore, we do not include them in our comparative analysis. Additionally, to demonstrate the comprehensive exploratory capability of NAS, we also include LT-Res in our comparisons. The experimental results are shown in Table 2. It is evident that the architectures we proposed exhibit heightened performance within a similar model size. Specifically, both LT-Res and LT-DARTS yielded superior results. Due to the more thorough exploration conducted by LT-DARTS, it consistently outperformed in all scenarios. The architecture discovered by LT-DARTS shows a 4% to 8% improvement over the basic ResNet baseline. Even when compared to more advanced architectures like ResNeXt and Res2Net, it still achieves at least a 1% accuracy advantage. Experiments across different datasets and imbalanced factors further validate the wide applicability of LT-DARTS.

5.2.2 Achieving SOTA Performance by Combining Existing Methods. Additionally, we compare our method with state-of-the-art (SOTA)

Table 2: We compare our approach with some state-of-the-art manually designed architectures on the CIFAR10/100-LT dataset. Without loss of generality, all networks are trained to converge using the cross-entropy loss.

Backbone	#P(M)	CIFAR-10-LT			CIFAR-100-LT		
		$\rho=200$	$\rho=100$	$\rho=50$	$\rho=200$	$\rho=100$	$\rho=50$
ResNet [16]	0.46	65.68 $\pm$ 0.25	70.70 $\pm$ 0.31	74.81 $\pm$ 0.17	34.84 $\pm$ 0.12	38.43 $\pm$ 0.09	43.90 $\pm$ 0.14
ResNeXt [40]	0.45	68.26 $\pm$ 0.34	74.14 $\pm$ 0.22	79.06 $\pm$ 0.27	36.44 $\pm$ 0.18	41.07 $\pm$ 0.11	46.61 $\pm$ 0.19
Res2Net [13]	0.45	67.92 $\pm$ 0.19	73.65 $\pm$ 0.14	78.23 $\pm$ 0.22	35.97 $\pm$ 0.32	40.28 $\pm$ 0.10	46.28 $\pm$ 0.21
LT-Res	0.45	69.35 $\pm$ 0.25	76.48 $\pm$ 0.20	81.44 $\pm$ 0.29	39.65 $\pm$ 0.20	41.55 $\pm$ 0.24	47.10 $\pm$ 0.18
LT-DARTS	0.46	70.23 $\pm$ 0.18	77.52 $\pm$ 0.14	82.45 $\pm$ 0.33	41.23 $\pm$ 0.22	42.31 $\pm$ 0.15	47.58 $\pm$ 0.21

Table 3: Comparison with SOTA methods. No specific annotation indicates using ResNet as the backbone.

	CIFAR-10-LT		CIFAR-100-LT	
	200	100	200	100
CE	65.68	70.70	34.84	38.43
CE+DARTS	62.31	67.29	31.46	36.02
CE+Ours	70.23	77.52	41.23	42.31
CE+Mixup [46]	65.84	72.96	35.84	40.01
CE+Mixup+DARTS	61.32	68.21	31.13	37.25
CE+Mixup+Ours	73.87	78.86	43.51	44.77
LDAM-DRW [5]	75.32	77.03	38.91	42.04
LDAM-DRW+DARTS	72.56	75.34	36.24	39.11
LDAM-DRW+Ours	77.26	79.61	44.45	45.79
Mixup+cRT [23]	73.06	79.15	41.73	45.12
Mixup+cRT+DARTS	70.97	76.45	38.05	42.39
Mixup+cRT+Ours	80.50	82.91	44.90	49.47
BBN [50]	73.47	79.82	37.21	42.56
BGP [35]	-	-	41.20	45.20
GCL [24]	79.03	82.68	44.88	48.71
Optimal	80.50	82.91	44.90	49.47

methods in the LT community. Our approach aims to find an optimal architecture, which is orthogonal to most existing solutions, making it easy to combine with various methods for further performance enhancement. Table 3 presents our experimental results. In each block of this table (except the last), the first row represents existing SOTA methods with a manually designed ResNet backbone, which is also the baseline. The second row shows SOTA methods combined with vanilla DARTS, and the third row features SOTA methods combined with our approach. We can conclude that: (i) Simply replacing the ResNet architecture in the baseline with LT-DARTS (CE+Ours) achieves superior performance compared to the SOTA method (BBN). It is important to note that the network is trained using the cross-entropy loss and does not incorporate any advanced rebalancing techniques. (ii) After enhancing the search space and strategy, LT-DARTS is able to identify architectures better suited to LT data, significantly outperforming the baseline. (iii) Moreover, the architecture discovered by LT-DARTS surpasses those of meticulously designed LT strategies, only with simple data augmentation (Mixup) and classifier retraining strategies (cRT).

5.2.3 Class-wise Accuracy under Long-Tailed Distribution. To further verify how the proposed method improves the performance, we

conduct a detailed class-wise performance analysis on the CIFAR-10-LT dataset with $\rho = 100$. Following standard practice in long-tailed recognition research [51], we divide the samples into three disjoint groups based on the number of training samples per class: head classes, medium classes, and tail classes. We evaluate and report the top-1 accuracy within each group for several representative baselines and our proposed methods.

As shown in Table 4, LT-DARTS achieves consistent improvements across all three class splits. This indicates that the searched architecture enhances the overall feature representation quality, regardless of class frequency. More importantly, the performance gain is especially prominent in the tail classes. For example, LT-DARTS improves tail-class accuracy from 41.0% (ResNet) to 66.0%, yielding a relative improvement of over 60%. While the head class accuracy also improves (from 72.5% to 77.2%), the margin is relatively smaller. This disproportionate improvement in tail-class performance suggests that our architectural search method is particularly effective at enhancing the discriminative capacity for under-represented classes, without sacrificing the performance on frequent classes. These findings validate our hypothesis that architecture-level adaptation serves as a powerful and orthogonal approach to long-tailed learning.

Table 4: Class-wise accuracy breakdown on CIFAR-10-LT.

Backbone	Head	Medium	Tail	Overall
ResNet	72.5	58.0	41.0	70.70
ResNeXt	75.0	66.5	52.0	74.14
DARTS	74.2	66.0	55.0	73.65
LT-ResNet (ours)	<u>76.5</u>	<u>72.0</u>	<u>64.0</u>	<u>76.48</u>
LT-DARTS (ours)	77.2	73.5	66.0	77.25

5.2.4 Large-scale Datasets. We also validate the effectiveness of our method on several large-scale long-tailed benchmarks, including Places-LT, ImageNet-LT, and iNaturalist-LT. As shown in Table 5, LT-DARTS achieves competitive or superior top-1 accuracy on all datasets, despite having fewer parameters than the commonly used backbones. For example, on ImageNet-LT, LT-DARTS achieves 54.9% accuracy with only 22M parameters, outperforming DisAlign (53.4%) built on ResNeXt-50 with 23M parameters. Similarly, on Places-LT and iNaturalist-LT, our architecture achieves 40.5% and 71.0% respectively, with lower model complexity compared to ResNet-152 and ResNet-50 backbones.

Table 5: Top-1 Accuracy on Places-LT, ImageNet-LT, and iNaturalist-LT with different backbones.

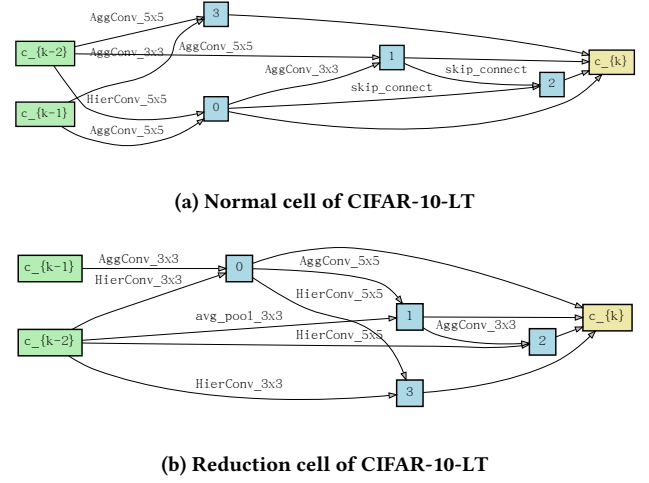
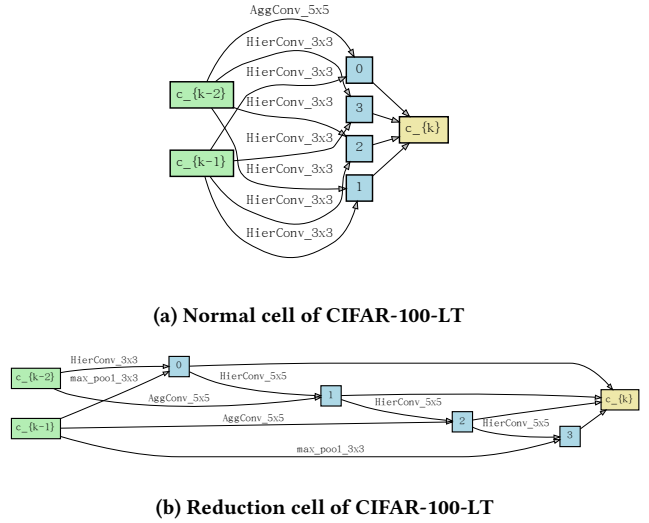
Method	Dataset	Backbone	#P(M)	Acc.
CE	Places-LT	ResNet-152	58.2	30.2
Decouple-cRT [23]		ResNet-152	58.2	36.7
Balanced Softmax [34]		ResNet-152	58.2	38.6
LADE [18]		ResNet-152	58.2	38.8
DisAlign [47]		ResNet-152	58.2	39.3
RSG [36]		ResNet-152	58.2	39.3
MiSLAS [49]		ResNet-152	58.2	40.2
GCL [24]		ResNet-152	58.2	40.6
Mixup+cRT		ResNet-152	58.2	38.5
Mixup+cRT		LT-DARTS	56.4	40.5
CE	ImageNet-LT	ResNeXt-50	23.0	44.4
Balanced Softmax [34]		ResNeXt-50	23.0	52.3
LADE [18]		ResNeXt-50	23.0	52.3
DisAlign [47]		ResNeXt-50	23.0	53.4
RSG [36]		ResNeXt-50	23.0	51.8
ResLT [9]		ResNeXt-50	23.0	52.9
Mixup+cRT		ResNeXt-50	23.0	52.3
Mixup+cRT		LT-DARTS	22.0	54.9
CE	iNaturalist-LT	ResNet-50	25.6	61.7
Balanced Softmax [34]		ResNet-50	25.6	70.6
LADE [18]		ResNet-50	25.6	70.0
DisAlign [47]		ResNet-50	25.6	70.6
RSG [36]		ResNet-50	25.6	70.3
ResLT [9]		ResNet-50	25.6	70.2
Mixup+cRT		LT-DARTS	24.8	71.0

Notably, these gains are obtained with minimal additional effort: our method is combined with simple long-tailed training techniques such as Mixup and cRT, without requiring complicated training pipelines, additional modules, or extensive hyperparameter tuning. Besides, it is important to emphasize that our goal is not to compete purely for SOTA performance. Instead, we focus on showcasing the potential of architecture design as a clean and principled approach that improves model performance across various long-tailed setting.

5.3 Architecture Analysis of Searched Cells

To better understand the structural patterns favored by LT-DARTS in LT scenarios, we analyze the architectures it discovers on CIFAR-10-LT and CIFAR-100-LT (see Fig. 10 and Fig. 11). Across both datasets, the searched cells consistently prefer LT-HierConv and LT-AggConv. This suggests that the two custom operations are particularly effective for addressing class imbalance. In addition, the discovered architectures incorporate a mix of kernel sizes (e.g., 3×3 and 5×5), skip connections, and moderate pooling operations. These design choices facilitate multi-scale feature extraction and help stabilize training, which is crucial under LT conditions.

Compared to manually designed networks, LT-DARTS demonstrates two key advantages: more suitable operation and greater structural flexibility. The proposed LT-HierConv and LT-AggConv integrate empirically favorable components, allowing the network to extract the feature of both head and tailed classes well. Further, LT-DARTS benefits from a search-driven design that adaptively assembles heterogeneous operations, while the hand-designed networks constrained by fixed topologies and homogeneous block structures.

**Figure 10: Cells with optimal performance searched on the CIFAR-10-LT Dataset.****Figure 11: Cells with optimal performance searched on the CIFAR-100-LT Dataset.**

In conclusion, our approach enables the automatic design of flexible and adaptive neural architectures along both component and network levels, thereby improving generalization performance on tail classes without compromising accuracy on head classes. This marks a clear divergence from the fixed structures of traditional residual networks.

5.4 Benefits of the BFC

We evaluate the role of the BFC in our method. It comes as no surprise that BFC ensures fairness throughout the search process. Fig. 13 illustrates changes in the classifier's weights during the architecture search process. It is obvious that the weight norms

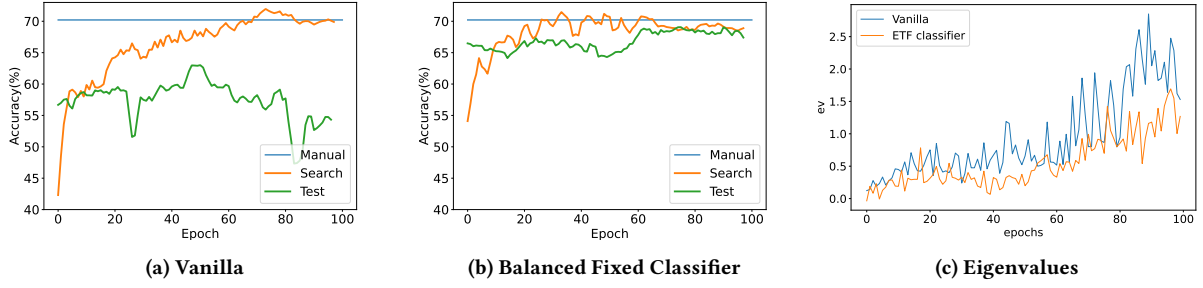


Figure 12: (a) and (b) show the performance differences between trainable and balanced fixed classifiers. “Search” indicates the search process performance and “test” result is a snapshot of the corresponding network training 100 epochs from scratch. (c) report the trajectory of Hessian matrix eigenvalues during architecture search.

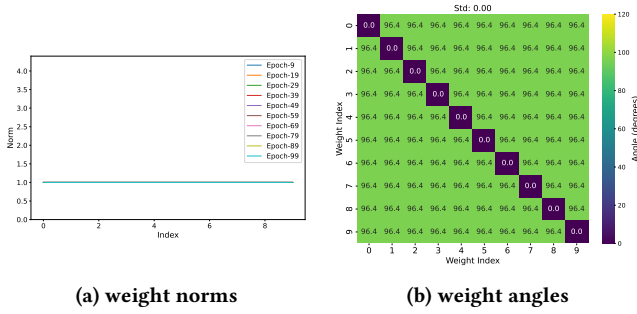


Figure 13: The weights of LT-DARTS. (a) is the weight norm during the search process and (b) represents the angle between pairwise weight vectors after the search is completed.

remain constant and the angles are equal, as this aligns with the derivation in Eq. (4).

Interestingly, we observe that the BFC appears to mitigate performance collapse [6, 30, 43–45], which is one of the key challenges in the DARTS field. As shown in Fig 12, the vanilla DARTS often suffers from a sharp performance decline in the later stages of architecture search, with architectures identified at these stages exhibiting notably inferior performance compared to those found earlier (see the green line in the figure). However, our approach ensures continuous improvement in architecture performance throughout the search process. Previous studies indicate that a smaller λ_{max}^α , which denotes dominant eigenvalue of architecture parameter α , can effectively mitigate performance collapse [6, 45]. Hence, we track the λ_{max}^α during the search process. As shown in Fig. 12c, the BFC achieves a smaller λ_{max}^α than the original method, providing compelling evidence that our approach successfully mitigates performance degradation and leading to improved overall search performance.

5.5 Ablation Study

We conduct ablation experiments to validate the improvements in both the search space and search strategy. We take the vanilla DARTS as the baseline and first improve its search space by adding

two novel convolution operations, namely LT-AggConv and LT-HierConv (LT-Conv). This modification enhances the architecture’s adaptability to tailed classes. Subsequently, we further improve the method by incorporating the BFC, aiming to reduce the negative impact of biased gradient updates during architecture search. The experimental results in Table 6 show that each component contributes positively to the final performance.

Table 6: Ablation studies on CIFAR-10-LT.

Vanilla DARTS	LT-Conv.	BFC	Acc.
✓			65.3
✓	✓		74.3
✓	✓	✓	77.8

6 Conclusion

Long-tailed (LT) data in real-world scenarios presents formidable challenges for recognition tasks. In this paper, we approach the issue from the overlooked perspective of architecture and bridge the gap between network architecture design and LT. We conduct comprehensive experiments to investigate which architectural components perform better under LT data. Based on our observations, we identify and design two high-performing candidate operations. To enable a more thorough and efficient exploration of architectural design, we propose LT-DARTS, a NAS method specifically tailored for LT scenarios. In addition to updating the search space, we introduce the balanced fixed classifier to address the biased search process under LT conditions. Extensive experiments demonstrate that our approach effectively designs LT-friendly architectures. Moreover, our method can integrate seamlessly with existing techniques and easily surpass state-of-the-art results, offering an orthogonal solution to LT challenges.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grant 62471451. The authors would like to thank the Information Science Laboratory Center of USTC for the hardware and software services.

GenAI Usage Disclosure

The authors used a LLM to assist with grammar checking and language polishing in the manuscript. The model was not used for generating technical content, code, data, or figures. All text was authored and fully reviewed by the human authors. The LLM served only as a writing aid, and no part of the final content is directly generated by AI.

References

- [1] Sumyeong Ahn, Jongwoo Ko, and Se-Young Yun. 2023. CUDA: Curriculum of Data Augmentation for Long-Tailed Recognition. *ArXiv abs/2302.05499* (2023).
- [2] Jimmy Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. 2016. Layer Normalization. *ArXiv abs/1607.06450* (2016).
- [3] Jianhong Bai, Zuozhu Liu, Hualiang Wang, Jinxiang Hao, Yang Feng, Huanpeng Chu, and Haoji Hu. 2023. On the Effectiveness of Out-of-Distribution Data in Self-Supervised Long-Tail Learning. *ArXiv abs/2306.04934* (2023).
- [4] Mateusz Buda, Atsuto Maki, and Maciej A. Mazurowski. 2017. A systematic study of the class imbalance problem in convolutional neural networks. *Neural networks: the official journal of the International Neural Network Society* 106 (2017), 249–259.
- [5] Kaidi Cao, Colin Wei, Adrien Gaidon, Nikos Aréchiga, and Tengyu Ma. 2019. Learning Imbalanced Datasets with Label-Distribution-Aware Margin Loss. In *Neural Information Processing Systems*.
- [6] Xiangning Chen and Cho-Jui Hsieh. 2020. Stabilizing Differentiable Architecture Search via Perturbation-based Regularization. In *International Conference on Machine Learning*.
- [7] Xiangxiang Chu, Tianbao Zhou, Bo Zhang, and Jixiang Li. 2019. Fair DARTS: Eliminating Unfair Advantages in Differentiable Architecture Search. *ArXiv abs/1911.12126* (2019).
- [8] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. 2016. Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs). *arXiv:1511.07289 [cs.LG]*
- [9] Jiequan Cui, Shu Liu, Zhuotao Tian, Zhisheng Zhong, and Jiaya Jia. 2021. ResLT: Residual Learning for Long-Tailed Recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45 (2021), 3695–3706.
- [10] Xuanyi Dong and Yi Yang. 2020. NAS-Bench-201: Extending the Scope of Reproducible Neural Architecture Search. *ArXiv abs/2001.00326* (2020).
- [11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xi-aohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2020. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ArXiv abs/2010.11929* (2020).
- [12] Jian Feng, Yajie He, Yuhan Pan, Zhipeng Zhou, Si Chen, and Wei Gong. 2024. Enhancing Fitness Evaluation in Genetic Algorithm-Based Architecture Search for AI-Aided Financial Regulation. *IEEE Transactions on Evolutionary Computation* 28, 3 (2024), 623–637.
- [13] Shanghua Gao, Ming-Ming Cheng, Kai Zhao, Xinyu Zhang, Ming-Hsuan Yang, and Philip H. S. Torr. 2019. Res2Net: A New Multi-Scale Backbone Architecture. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43 (2019), 652–662.
- [14] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. 2011. Deep Sparse Rectifier Neural Networks. In *International Conference on Artificial Intelligence and Statistics*.
- [15] Wei Gong, Linxiao Cao, Yifei Zhu, Fang Zuo, Xin He, and Haoquan Zhou. 2023. Federated Inverse Reinforcement Learning for Smart ICUs With Differential Privacy. *IEEE Internet of Things Journal* 10, 21 (2023), 19117–19124.
- [16] Kaiming He, X. Zhang, Shaoqing Ren, and Jian Sun. 2015. Deep Residual Learning for Image Recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2015), 770–778.
- [17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Identity Mappings in Deep Residual Networks. *arXiv:1603.05027 [cs.CV]*
- [18] Youngkyu Hong, Seungju Han, Kwanghee Choi, Seokjun Seo, Beomsu Kim, and Buru Chang. 2020. Disentangling Label Distribution for Long-tailed Visual Recognition. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2020), 6622–6632.
- [19] Y. Hong, Jianfu Zhang, Zhongyi Sun, and Ke Yan. 2022. SAFA: Sample-Adaptive Feature Augmentation for Long-Tailed Image Classification. In *European Conference on Computer Vision*.
- [20] Jie Hu, Li Shen, Samuel Albanie, Gang Sun, and Enhua Wu. 2017. Squeeze-and-Excitation Networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (2017), 2011–2023.
- [21] Sergey Ioffe and Christian Szegedy. 2015. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *ArXiv abs/1502.03167* (2015).
- [22] Xinyuan Ji, Zhaowei Zhu, Wei Xi, Olga Gadyatskaya, Zilong Song, Yong Cai, and Yang Liu. 2024. FedFixer: Mitigating heterogeneous label noise in federated learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38, 12830–12838.
- [23] Bingyi Kang, Saining Xie, Marcus Rohrbach, Zhicheng Yan, Albert Gordo, Jiashi Feng, and Yannis Kalantidis. 2019. Decoupling Representation and Classifier for Long-Tailed Recognition. *ArXiv abs/1910.09217* (2019).
- [24] Mengke Li, Yiu ming Cheung, and Yang Lu. 2021. Long-tailed Visual Recognition via Gaussian Clouded Logit Adjustment. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021), 6919–6928.
- [25] Shuang Li, Kaixiong Gong, Chi Harold Liu, Yulin Wang, Feng Qiao, and Xin-jing Cheng. 2021. MetaSAug: Meta Semantic Augmentation for Long-Tailed Visual Recognition. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021), 5208–5217.
- [26] Bo Liu, Haoxiang Li, Hao Kang, Gang Hua, and Nuno Vasconcelos. 2021. GistNet: a Geometric Structure Transfer Network for Long-Tailed Recognition. *2021 IEEE/CVF International Conference on Computer Vision (ICCV)* (2021), 8189–8198.
- [27] Hanxiao Liu, Karen Simonyan, and Yiming Yang. 2018. DARTS: Differentiable Architecture Search. *ArXiv abs/1806.09055* (2018).
- [28] Shenghao Liu, Guoyang Wu, Xianjun Deng, Hongwei Lu, Bang Wang, Laurence Yang, and James J. Park. 2023. Graph sampling based fairness-aware recommendation over sensitive attribute removal. In *2023 IEEE International Conference on Data Mining (ICDM)*. IEEE, 428–437.
- [29] Diganta Misra. 2019. Mish: A Self Regularized Non-Monotonic Neural Activation Function. *ArXiv abs/1908.08681* (2019).
- [30] Sajad Movahedi, Melika Adabinejad, Ayyoob Imani, Arezou Keshavarz, Mostafa Dehghani, Azadeh Shakery, and Babak Nadjar Araabi. 2022. A-DARTS: Mitigating Performance Collapse by Harmonizing Operation Selection among Cells. *ArXiv abs/2210.07998* (2022).
- [31] Gi Cheon Nam, Sunguk Jang, and Juho Lee. 2023. Decoupled Training for Long-Tailed Classification With Stochastic Representations. *ArXiv abs/2304.09426* (2023).
- [32] Yuhan Pan, Zhipeng Zhou, Wei Gong, and Yuguang Fang. 2024. SAT: A Selective Adversarial Training Approach for WiFi-Based Human Activity Recognition. *IEEE Transactions on Mobile Computing* 23, 12 (2024), 12706–12716.
- [33] Seulki Park, Youngkyu Hong, Byeongho Heo, Sangdoo Yun, and Jin Young Choi. 2021. The Majority Can Help the Minority: Context-rich Minority Oversampling for Long-tailed Classification. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021), 6877–6886.
- [34] Jiawei Ren, Cunjun Yu, Shunan Sheng, Xiao Ma, Haiyu Zhao, Shuai Yi, and Hongsheng Li. 2020. Balanced Meta-Softmax for Long-Tailed Visual Recognition. *ArXiv abs/2007.10740* (2020).
- [35] D. Wang, Yicheng Liu, Liangji Fang, Fanhua Shang, Yuanyuan Liu, and Hongying Liu. 2022. Balanced Gradient Penalty Improves Deep Long-Tailed Learning. *Proceedings of the 30th ACM International Conference on Multimedia* (2022).
- [36] Jianfeng Wang, Thomas Lukasiewicz, Xiaolin Hu, Jianfei Cai, and Zhenghua Xu. 2021. RSG: A Simple but Effective Module for Learning Imbalanced Datasets. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021), 3783–3792.
- [37] Shuai Wang, Tian He, Desheng Zhang, Yuanhao Shu, Yunhui Liu, Yu Gu, Cong Liu, Haengju Lee, and Sang H Son. 2018. BRAVO: Improving the rebalancing operation in bike sharing with rebalancing range prediction. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 2, 1 (2018), 1–22.
- [38] Tong Wu, Ziwei Liu, Qingqiu Huang, Yu Wang, and Dahua Lin. 2021. Adversarial Robustness under Long-Tailed Distribution. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021), 8655–8664.
- [39] Yuxin Wu and Kaiming He. 2018. Group Normalization. *International Journal of Computer Vision* 128 (2018), 742–755.
- [40] Saining Xie, Ross B. Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. 2016. Aggregated Residual Transformations for Deep Neural Networks. *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016), 5987–5995.
- [41] Yuhui Xu, Lingxi Xie, Xiaopeng Zhang, Xin Chen, Guo-Jun Qi, Qi Tian, and Hongkai Xiong. 2019. Pc-darts: Partial channel connections for memory-efficient architecture search. *arXiv preprint arXiv:1907.05737* (2019).
- [42] Yibo Yang, Shixiang Chen, Xiangtai Li, Liangru Xie, Zhouchen Lin, and Dacheng Tao. 2022. Inducing Neural Collapse in Imbalanced Learning: Do We Really Need a Learnable Classifier at the End of Deep Neural Network?. In *Neural Information Processing Systems*.
- [43] Peng Ye, Tong He, Baopu Li, Tao Chen, Lei Bai, and Wanli Ouyang. 2023. β -DARTS++: Bi-level Regularization for Proxy-robust Differentiable Architecture Search. *ArXiv abs/2301.06393* (2023).
- [44] Peng Ye, Baopu Li, Yikang Li, Tao Chen, Jiayuan Fan, and Wanli Ouyang. 2022. β -DARTS: Beta-Decay Regularization for Differentiable Architecture Search. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2022), 10864–10873.
- [45] Arber Zela, Thomas Elsken, Tonmoy Saikia, Yassine Marrakchi, Thomas Brox, and Frank Hutter. 2019. Understanding and Robustifying Differentiable Architecture

- Search. *ArXiv abs/1909.09656* (2019).
- [46] Hongyi Zhang, Moustapha Cissé, Yann Dauphin, and David Lopez-Paz. 2017. mixup: Beyond Empirical Risk Minimization. *ArXiv abs/1710.09412* (2017).
 - [47] Songyang Zhang, Zeming Li, Shipeng Yan, Xuming He, and Jian Sun. 2021. Distribution Alignment: A Unified Framework for Long-tail Visual Recognition. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021), 2361–2370.
 - [48] Ye Zhang and Byron C. Wallace. 2015. A Sensitivity Analysis of (and Practitioners' Guide to) Convolutional Neural Networks for Sentence Classification. *ArXiv abs/1510.03820* (2015).
 - [49] Zhisheng Zhong, Jiequan Cui, Shu Liu, and Jiaya Jia. 2020. Improving Calibration for Long-Tailed Recognition. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2020), 16484–16493.
 - [50] Boyan Zhou, Quan Cui, Xiu-Shen Wei, and Zhao-Min Chen. 2019. BBN: Bilateral-Branch Network With Cumulative Learning for Long-Tailed Visual Recognition. *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2019), 9716–9725.
 - [51] Zhipeng Zhou, Lanqing Li, Peilin Zhao, Pheng-Ann Heng, and Wei Gong. 2023. Class-Conditional Sharpness Aware Minimization for Deep Long-Tailed Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*.
 - [52] Zhipeng Zhou, Feng Wang, Jihong Yu, Ju Ren, Zhi Wang, and Wei Gong. 2022. Target-oriented Semi-supervised Domain Adaptation for WiFi-based HAR. In *IEEE INFOCOM 2022 - IEEE Conference on Computer Communications*. 420–429.